

1. Problema

Uma instituição financeira digital precisa decidir, em diferentes canais, qual oferta, mensagem ou próximo passo apresentar para cada cliente elegível. Regras fixas e testes A/B longos desperdiçam tráfego e demoram a reagir a mudanças de contexto. Este projeto resolve o problema com uma abordagem de multi-armed bandit contextual: identificar comportamentos distintos por segmento, equilibrar exploração e exploração, e aprender com respostas observadas sem congelar a decisão em regras estáticas – incluindo um assistente com LLM que resume experimentos, recupera políticas internas sintéticas e explica decisões.

2. Base de dados escolhida

Kaggle "bank-marketing" (henriqueyamahata, licença CC0), derivada do UCI Bank Marketing Dataset (Moro, Cortez & Rita, 2014) – campanhas de telemarketing bancário português, 2008-2013, 41.188 linhas, 21 colunas processadas. A coluna `duration` foi removida por vazamento temporal (só é conhecida após o fim da ligação). Valores sentinela `unknown` existem em 6 colunas categóricas (destaque: `default`, 20,9%). O alvo é desbalanceado (88,7% "no" / 11,3% "yes", ~7,9:1). Detalhes completos em `reports/data-quality.md` e `data/kaggle/README.md`.

3. Enriquecimento sintético

Uma camada de experimentação adaptativa foi construída sobre a base Kaggle, fisicamente separada dela (`data/synthetic\_enrichment/`): catálogo de ofertas/braços, eventos de impressão com contexto de decisão, e recompensas atrasadas (delayed rewards) com horizonte temporal documentado. Sementes aleatórias controladas garantem reprodutibilidade. Documentos de política comercial sintéticos (`policy\_docs/`) alimentam tanto o guard de suitability quanto a base de conhecimento do assistente RAG. Processo completo em `reports/data-generation.md`.

4. Modelagem como multi-armed bandit

Três políticas foram implementadas e comparadas:

Política	Papel	Notas de implementação
Baseline determinístico	Controle	Melhor braço histórico por segmento (`fit` sobre a tabela de treino)
Thompson Sampling	Exploração bayesiana	Beta-Bernoulli por (segmento, braço); priors de warm-start via um modelo de propensão em PyTorch
LinUCB	Família UCB	Ridge regression disjunta por braço + bônus de confiança (Li et al., 2010)

A referência "Nilos-UCB" do edital não foi localizada na literatura acadêmica disponível; o time adotou LinUCB como resposta à família UCB solicitada, com a justificativa completa registrada em `reports/algorithm-comparison.md`. O contexto de decisão entra via um vetor de 16 features (12 one-hot de `job`, idade normalizada, 3 one-hot de `outcome`). Recompensas atrasadas são tratadas via a tabela de treino sintética, que já une eventos a seus resultados finais antes do replay.

5. Comparação quantitativa

Avaliação offline via replay-with-rejection (Li et al., 2011), válida porque a política de logging sintética é uniforme aleatória:

Política	Regret médio/decisão	Decisões aceitas	Entropia de seleção
baseline	0.026769	3.318	1.0691
thompson_sampling	0.024970	3.520	1.5349
linucb	0.029457	3.343	1.6940

Golden set (22 casos, 4 categorias): todas as três políticas atingem 100% de taxa de segurança (`passed\_safety`); acurácia vs. ação esperada do oráculo é 18,2% (baseline), 13,6% (thompson_sampling), 18,2% (linucb). Fairness de exposição por segmento: mínimo 13,6%, máximo 18,7%, contra 16,7% teórico uniforme, em 72 células (12 segmentos × 6 braços). Detalhes e análise de sensibilidade de hiperparâmetros em `reports/offline-evaluation.md`.

6. Arquitetura-alvo Azure

Arquitetura exclusivamente Azure (Container Apps consumption com ``min_replicas=0``, Blob Storage Standard/LRS, Key Vault + Managed Identity, Log Analytics + Application Insights), com Terraform completo em ``infra/terraform/`` (``init`/`validate`` reais; ``apply`` deliberadamente não executado – sem credenciais Azure configuradas ainda). Custo estimado: ~US\$5-10/mês, com um guardrail real de orçamento (``azurerms_consumption_budget_resource_group``, US\$20/mês, alerta em 80%). Decisões de custo: GitHub Container Registry (gratuito) em vez de Azure Container Registry (~US\$5/mês fixo); Container Apps consumption em vez de App Service dedicado (~US\$13/mês mesmo ocioso). Diagrama completo e mapeamento de camadas em ``docs/architecture-azure.md``.

7. Ciclo de vida MLOps

Registro de políticas versionado (``PolicyRegistry``) com estados ``pending_approval`` → ``approved`` → ``rejected``, critérios de promoção objetivos (`safety rate` $\geq 100\%$, `regret absoluto` ≤ 0.03 , `regressão vs. política ativa` $\leq 10\%$), gate de aprovação humana estruturada (``bandit-cli approve``, nunca promove sem aprovação registrada), rollback com histórico, monitoramento de drift de features (PSI) e de performance entre versões, e rastreamento de experimentos em MLflow local. Um passeio guiado real (não simulado) está documentado em ``docs/mlops-lifecycle.md``, cobrindo três hipóteses: uma formalização da política de produção atual (aprovada), uma rejeitada automaticamente e aprovada apenas via override humano explícito (sem chegar a ser promovida), e uma promovida e depois revertida via rollback.

8. Limitações

O guard de suitability codificado cobre apenas duas das regras de negócio documentadas em ``data/synthetic_enrichment/policy_docs/`` (falta, entre outras, "não repetir oferta recusada" e "validade de 30 dias da promoção" – ``reports/offline-evaluation.md`` §7). O golden set é curado, não estatisticamente amostrado. A análise de sensibilidade testou apenas 6 combinações de hiperparâmetros. Não existe hoje um loop de feedback de recompensa em tempo real – o monitoramento de "recompensa" é feito por comparação de regret médio entre versões no momento do retreino, não por decisão individual (ver ``docs/system-card.md``). O cache de política ativa é por processo – uma promoção só é refletida por um serviço já em execução após reinício.

9. Riscos

Os quatro cenários de risco formais (reward hacking, manipulação de contexto, abuso do assistente, violação de suitability) estão detalhados em ``docs/system-card.md``. O mais relevante hoje é a violação de suitability: a lacuna documentada entre regras de negócio e código é um bloqueio real antes de qualquer uso além de demonstração.

10. Hipóteses e trabalhos futuros

Hipóteses testáveis para uma próxima iteração: (1) completar a cobertura das regras de suitability ainda não codificadas (ver §8); (2) conectar um sinal de recompensa real com validação de qualidade antes de qualquer retreino automático; (3) agendar ``monitor-drift`` automaticamente em vez de execução manual; (4) migrar a leitura de ``ANTHROPIC_API_KEY`` de ``env`` para Key Vault via Managed Identity, completando o desenho já documentado em ``docs/architecture-azure.md``; (5) expandir o golden set com amostragem estatística complementar à curadoria manual atual.

11. Referências

- Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). *A Contextual-Bandit Approach to Personalized News Article Recommendation.* WWW 2010.
- Li, L., Chu, W., Langford, J., & Wang, X. (2011). *Unbiased Offline Evaluation of Contextual-bandit-based News Article Recommendation Algorithms.* WSDM 2011.
- Moro, S., Cortez, P., & Rita, P. (2014). *A Data-Driven Approach to Predict the Success of Bank Telemarketing.* Decision Support Systems.
- Dataset: Kaggle "bank-marketing" (henriqueyamahata), licença CC0.
- Lei nº 13.709/2018 (LGPD).
- Documentação MLflow, Terraform ``azurerm`` provider, FastAPI, Streamlit.